

SENTRA: Privacy-Preserving Training in Outsourced Cloud Environments

Maryam Zarezadeh
Barkhausen Institut
Dresden, Germany
maryam.zarezadeh@barkhauseninstitut.org

Jana Eisoldt
Barkhausen Institut
Dresden, Germany
jana.eisoldt@barkhauseninstitut.org

Bhavish Mohee
University of Sheffield
Sheffield, United Kingdom
bmohee1@sheffield.ac.uk

Stefan Köpsell
Barkhausen Institut
Dresden, Germany
stefan.koepsell@barkhauseninstitut.org

Behzad Abdolmaleki
University of Sheffield
Sheffield, United Kingdom
behzad.abdolmaleki@sheffield.ac.uk

Abstract—Training machine learning models in untrusted clouds requires strong guarantees of confidentiality, integrity, and correctness, while remaining scalable and resilient to node churn. These challenges are further amplified in emerging agentic AI systems, where autonomous and distributed learning components require trustworthy coordination and secure state management across heterogeneous cloud environments. Existing Trusted Execution Environments (TEEs) lack scalability and remain vulnerable to side-channel attacks for large workloads, while pure secure multi-party computation (MPC) approaches incur prohibitive overhead in practice. SENTRA (Secure ENclave-based TRaining Architecture) addresses these challenges through a hybrid architecture that combines TEEs, secret sharing, and communication-efficient MPC with system-level mechanisms that secure the entire training lifecycle. SENTRA introduces a scalable collective attestation protocol that verifies all participating enclaves and enforces hardware exclusivity before any node may store or process secret shares. Training data and model parameters are stored as secret shares in a versioned enclave-backed key-value store (KVS), providing rollback protection and consistent state under adversarial conditions. SENTRA further supports dynamic, fault-tolerant membership through Dynamic Proactive Secret Sharing (DPSS)-based resharing, safe packed-MPC computation under degree bounds, and adaptive handling of node failures. Evaluation of a prototype implementation shows that SENTRA achieves up to 8.89 samples/s throughput and $1.29\times$ faster training than the CryptTen baseline in software-only mode. In hardware-enclave mode, SENTRA incurs only an 8.3% performance overhead while providing memory-isolated confidentiality, fault-tolerant membership management, rollback protection, and recovery from node failures in approximately 8 seconds.

Index Terms—Trusted Execution Environment, MPC, privacy-preserving ML, Cloud, Training, KVS, TEE.

1. Introduction

The widespread adoption of cloud computing has transformed the training and deployment of large-scale machine learning (ML) models. Public cloud platforms provide elastic scalability and distributed services, making them the dominant environment for deep neural network (DNN) training. Recent advances in agentic AI systems further increase the importance of secure and trustworthy distributed training infrastructure. Autonomous and multi-agent AI systems increasingly rely on cloud-hosted coordination, distributed state management, and continuous model updates across heterogeneous environments. In such settings, compromise of training data, model state, or coordination logic may propagate across distributed learning components, amplifying the impact of confidentiality breaches, rollback attacks, and unauthorized computation. As a result, agentic systems require strong guarantees of integrity, freshness, and resilient execution in addition to scalable training performance. However, outsourcing training to third-party providers introduces major security and privacy risks, especially when datasets contain sensitive information such as medical records, financial data, or proprietary enterprise assets.

A key challenge is protecting data during computation. Although encryption at rest and in transit is standard, securing data in use remains difficult when the cloud provider and software stack are not fully trusted. Confidential computing addresses this issue through hardware-based Trusted Execution Environments (TEEs) [1], [2]. TEEs such as Intel SGX, Intel TDX, and AMD SEV-SNP isolate sensitive code and data from the host system and support remote attestation to verify execution integrity before provisioning secrets [2], [3]. However, TEEs alone are insufficient for scalable distributed training due to limited protected memory, enclave overhead, and vulnerability to side-channel attacks [4], [5]. In addition, single-enclave designs create concentration risk, where compromising one enclave may expose the entire dataset or model state. Cryptographic methods based on Secure Multi-Party Computation (MPC) provide a comple-

mentary approach. Threshold secret sharing schemes such as Shamir Secret Sharing (SSS) [6] and packed variants [7] enable computation on private data without revealing plaintext to individual participants. Recent MPC frameworks have demonstrated privacy-preserving neural network inference and training [8]–[10], though they often incur high communication and computation overhead for large-scale models. Beyond confidentiality, outsourced training must also handle distributed system challenges such as node failures, churn, and consistency management. Distributed storage systems like Dynamo [11] achieve scalability through replication and partitioning, while remote attestation in decentralized settings remains an active research problem [12], [13]. Overall, neither TEEs nor MPC alone fully satisfy the requirements of confidential, scalable, and fault-tolerant deep learning in untrusted cloud environments. A practical solution requires integrating hardware-backed isolation, threshold cryptography, distributed storage, and robust membership control to ensure confidentiality, computation integrity, resilience to node churn, and protection against rollback and replay attacks.

1.1. Problem Statement

Despite significant progress in confidential computing and privacy-preserving machine learning, existing systems address these challenges only partially. TEE-based frameworks mainly protect computation within single enclaves or small trusted clusters, but often assume static membership and lack mechanisms for distributed consistency, rollback protection, and secure reconfiguration under node churn. In contrast, MPC-based systems provide strong cryptographic privacy guarantees, yet frequently overlook practical cloud concerns such as dynamic resource allocation, heterogeneous hardware, and adversarial control of the software stack. Similarly, prior work on remote attestation verifies execution integrity, but does not integrate attestation with secret-shared storage, versioned model state, and iterative deep learning workflows. Consequently, a gap remains between cryptographic security models and the operational requirements of distributed deep learning in untrusted clouds. No existing system simultaneously provides (i) hardware-backed protection of data in use, (ii) threshold-based confidentiality across nodes, (iii) rollback-resistant distributed storage, and (iv) safe dynamic membership with formal safety guarantees for packed computation. Addressing this gap requires a co-designed approach that tightly integrates attestation, secret sharing, storage semantics, and training execution. Training deep neural networks in untrusted public clouds therefore raises challenges in *security*, *correctness*, and *scalability*. Sensitive training data and model parameters must remain confidential, yet TEEs such as Intel SGX and TDX suffer from limited memory, scalability constraints, and side-channel vulnerabilities, making them insufficient for large-scale workloads. Pure MPC approaches, while privacy-preserving, incur substantial computation and communication overhead for deep learning. These challenges intensify in realistic deployments where nodes may join,

leave, or fail during training, requiring secure redistribution of secrets and continuous correctness guarantees. Preventing *rollback and replay attacks* is also critical to ensure model freshness under a strong adversary controlling the cloud software stack. At the same time, practical systems must achieve performance close to plaintext training while preserving confidentiality and integrity. **SENTRA (Secure ENclave-based TRaining Architecture)** addresses these issues by combining hardware-backed execution, cryptographic secret sharing, and distributed coordination under a *strong attestation requirement*: only verified and up-to-date enclaves may store or process secret shares.

Objectives. The goal of SENTRA is to enable *secure, scalable, and efficient* neural network training in untrusted clouds. SENTRA is designed to:

- preserve training-data confidentiality against cloud providers, co-tenants, and external adversaries,
- protect model parameters, gradients, and intermediate state from disclosure and tampering,
- combine TEEs with Shamir-based secret sharing to overcome the limitations of either technique in isolation,
- support efficient large-scale distributed training without prohibitive overhead,
- remain resilient to node churn (dynamic joins, leaves, and failures),
- prevent rollback and enforce freshness of model and dataset versions, and
- defend against a strong adversary controlling the cloud software stack and network.

Why not a client-only design? A natural alternative is to keep the dataset at the client while the cloud performs encrypted computation. However, this is impractical because: (i) clients cannot remain involved across millions of SGD iterations; (ii) HE or client-key MPC requires continuous client availability or incurs prohibitive overhead; (iii) rollback protection requires distributed state, which client-only storage cannot provide; (iv) cloud failures would require retransmitting datasets; and (v) compromising a single TEE could expose all data, whereas secret sharing limits leakage to meaningless shares. Thus, client-only approaches introduce bottlenecks, rollback risks, and single points of failure.

The need for attestation. Since the cloud platform itself is untrusted, confidentiality and correctness depend on ensuring that all participating nodes: (i) execute approved code inside genuine TEEs, (ii) provide fresh attestation evidence, and (iii) are not silently replaced, downgraded, or co-located in ways that violate system assumptions. Without robust attestation, a malicious provider could inject unauthorized nodes, replay stale attestations, or bypass diversity guarantees, undermining both secret sharing and computation integrity.

Attestation as the central challenge. SENTRA addresses this through a scalable *collective attestation* mechanism that establishes the *integrity and authenticity* of all participating enclaves while ensuring freshness of trust evidence. Specifically, collective attestation: (i) verifies that nodes execute approved code inside genuine TEEs and (ii) binds fresh attestation results to a membership

epoch. These attestation results form the trust anchor for higher-level coordination mechanisms. Based on attested identities, SENTRA: (iii) regulates secure node admission and removal, (iv) enforces hardware diversity constraints during committee formation, and (v) binds access credentials and key distribution to the current attested membership. Together with secret sharing, versioned storage, and Dynamic Proactive Secret Sharing (DPSS) resharing [14], these mechanisms provide confidentiality, computation integrity, rollback protection, and safe dynamic membership in untrusted cloud environments.

1.2. Our Contribution

This work introduces *SENTRA*, a system that enables *efficient, confidential, and fault-tolerant training of deep neural networks in untrusted cloud environments*. SENTRA overcomes the limitations of approaches that rely solely on TEEs or solely on MPC by combining hardware-assisted secure execution, communication-efficient secret sharing, and robust distributed coordination mechanisms. Our key contributions are as follows:

- i) *Attestation for secure and dynamic membership*. SENTRA introduces an attestation mechanism that verifies all participating enclaves before they may store secret shares or participate in MPC computation. It prevents stale, downgraded, or co-located enclaves from joining the system and enables secure node inclusion during dynamic membership changes.
- ii) *Confidential storage and privacy-preserving training*. Training data and model parameters are secret-shared and stored inside hardware enclaves. No single node can reconstruct sensitive values, providing end-to-end confidentiality during storage, communication, and training.
- iii) *Dynamic environment support*. SENTRA tolerates node churn by combining a secure Node-Join protocol with DPSS [14]. New nodes can be added, failed nodes replaced, and missing shares restored without reconstructing plaintext, enabling uninterrupted and autonomous training.
- iv) *Fault tolerance and safe packed-MPC computation*. SENTRA ensures continuous progress despite node failures by enforcing the Packed Shamir Secret Sharing (PSS) [7] safety bound $2(t + s - 1) < n_{\text{active}}$, where t is the adversarial threshold, s the packing factor, and n_{active} the number of currently live nodes. Within this bound, training proceeds safely; if the bound is violated, SENTRA dynamically reduces the packing factor or invokes DPSS to refresh shares. The maximum number of tolerated failures is $(f_{\text{max}} = n - (2(t + s - 1) + 1))$. Training pauses only when failures exceed this threshold, then automatically resumes once safety is restored.
- v) *Distributed design for side-channel resilience*. Unlike single-TEE systems, SENTRA distributes sensitive computation and storage across multiple enclaves. Compromise of one enclave reveals only a single share, significantly reducing the impact of side-channel attacks and strengthening robustness against a malicious cloud

provider. Additionally, SENTRA is hardware independent, allowing nodes to use different secure hardware in a training epoch. The usage of heterogeneous secure hardware across participating nodes enhances system robustness, as it complicates adversarial efforts by requiring the exploitation of diverse hardware-specific attack surfaces.

- vi) *Prototype implementation and evaluation*. We implement and evaluate SENTRA as an end-to-end privacy-preserving training system. Our results demonstrate practical feasibility of confidential training with rollback protection and secure dynamic membership. We also demonstrated that the overhead introduced by using TEEs is within an expected [15] and acceptable range of roughly 30%.

1.3. Technical Overview

SENTRA enables confidential, scalable, and fault-tolerant training of deep neural networks in untrusted cloud environments (see Appendix A.1 for detailed training procedures). It achieves this by tightly integrating hardware-backed TEEs, threshold secret sharing, communication-efficient MPC, and distributed coordination mechanisms. The core design principle is that no single technique alone is sufficient: TEEs provide isolation but lack scalability, while pure MPC provides strong cryptographic privacy but incurs prohibitive overhead. SENTRA combines both approaches to obtain practical security and performance.

Execution Flow. At a high level, SENTRA operates in recurring membership epochs. Each epoch consists of collective attestation, committee selection, secure storage and computation, and optional resharing if membership changes occur.

- 1) *Collective Attestation*. All participating nodes produce fresh TEE attestation evidence bound to an epoch nonce. A verifier quorum validates enclave measurements and aggregates the results into a collective attestation certificate. Only successfully attested nodes are eligible to participate in storage or computation.
- 2) *Committee Formation*. From the set of eligible nodes, a committee for the next epoch is selected using trust-weighted pseudorandom ordering under diversity constraints (hardware platform, operator, physical host). This prevents concentration of trust in a single provider and mitigates correlated failures or side-channel exposure.
- 3) *Secure Data Ingestion and Storage*. Training data and model parameters are encoded using Shamir or Packed Shamir Secret Sharing and distributed across enclave-backed KVS shards. Each shard stores only a secret share together with monotonically increasing version metadata, preventing rollback or replay attacks.
- 4) *Packed MPC Training*. Mini-batches are retrieved via version-aware reads. Standard Shamir shares are converted PSS to amortize communication and enable SIMD-style secure linear algebra. Forward and backward passes, gradient computation, and parameter up-

dates are executed collaboratively inside TEEs using communication-efficient MPC.

- 5) *Versioned Updates and Freshness*. After each training iteration, updated model shares are written back to the KVS under a new version number. Compare-and-set semantics enforced inside enclaves guarantee monotonicity and prevent stale writes. An update is committed only after a quorum of shards acknowledges the new version.
- 6) *Dynamic Membership and Resharing*. Node joins, failures, or departures trigger DPSS resharing. Shares are refreshed or redistributed without reconstructing plaintext, and a new epoch is activated. Old credentials and shares are invalidated, ensuring that revoked nodes cannot access updated secrets.

Fault Tolerance and Safety Bound. SENTRA maintains correctness under node churn by enforcing the packed-MPC safety condition $2(t + s - 1) < n_{\text{active}}$, where t is the adversarial threshold, s is the packing factor, and n_{active} is the number of currently live nodes. If the bound becomes tight, the system reduces the packing factor. If it is violated, training pauses and resharing or node replacement is initiated. This ensures that polynomial degrees remain decodable and that privacy and correctness guarantees are preserved.

Application of SENTRA. We demonstrate SENTRA in a collaborative medical training scenario (Fig. 1). Hospitals upload patient data as secret shares to enclave-backed storage nodes, ensuring that raw records are never exposed and no single node can reconstruct the data. Training runs across attested nodes using packed MPC within enclaves. Nodes retrieve shares, perform forward and backward passes, and compute model updates without revealing plaintext values. Updates are stored in a distributed KVS, where versioning provides rollback protection and quorum mechanisms ensure model freshness and consistency. This design protects sensitive medical data, enables autonomous training, tolerates node churn through resharing, and resists rollback and tampering through version-controlled updates and attested execution.

2. Preliminaries

2.1. Definitions

Definition 1 (Shamir Secret Sharing (SSS)). SSS [6] divides a secret $s \in \mathbb{F}$ among n parties such that any subset of at least $t+1$ parties can reconstruct the secret, while any subset of at most t parties learns nothing about it. A random polynomial f of degree t is chosen with $f(\alpha_0) = s$, and each participant P_i receives the share $f(\alpha_i) \in \mathbb{F}$, where $\alpha_0, \alpha_1, \dots, \alpha_n$ are fixed field elements.

Definition 2 (Packed Secret Sharing (PSS)). PSS [7] extends Shamir Secret Sharing by encoding multiple secrets within a single polynomial. To share ℓ secrets $(s_1, \dots, s_\ell) \in \mathbb{F}^\ell$, a polynomial f of degree t is chosen

such that $f(\alpha_0) = s_1, f(\alpha_{-1}) = s_2, \dots, f(\alpha_{-(\ell-1)}) = s_\ell$. Each participant P_i receives the share $f(\alpha_i) \in \mathbb{F}$. The scheme guarantees privacy against up to $t_p = t - \ell$ corrupted parties, while any $t+1$ parties can reconstruct all ℓ secrets.

Definition 3. A Trusted Execution Environment (TEE) is a secure processor-isolated environment that protects sensitive code and in-use data with confidentiality and integrity guarantees, enabling confidential computing [1]; see Definition 7 in Appendix A.

Definition 4 (Key Value Store (KVS)). KVSs [11] are NoSQL storage systems widely used for cloud applications, as they offer efficient retrieval of key-value pairs. A *distributed* KVS [16] has the following advantages: a) scalability: via horizontal scaling, increasing traffic can be handled b) reliability and availability: due to the use of replication. A challenge of distributed systems is consistency, as updates take time to propagate through the system.

Definition 5. Highest Random Weight (HRW) hashing [17], also called Rendezvous Hashing, consistently assigns objects to distributed nodes by computing a hash-based weight for each node and selecting the node with the highest weight. HRW provides load balancing, scalability, and minimal data movement when nodes join or leave.

Definition 6 (Dynamic-committee Proactive Secret Sharing (DPSS)). DPSS [14] is a technique to transfer secret shares from a set of committee C to a potentially different committee C' . With the event of *asynchronous* DPSS [14], the protocol remains robust if nodes of the committee become unavailable.

3. Related Work

3.1. Remote Attestation in Distributed Environments

Traditional remote attestation relies on a single attester and verifier, but large-scale IoT and cloud systems require distributed approaches. IoT research explores collective and non-interactive attestation under resource constraints [18], [19], while cloud systems provide stronger security and privacy guarantees. Recent work supports mutual attestation without trusted third parties and across heterogeneous systems [20], [21]. zRA [22] uses zkSNARKs for selective disclosure in attestation, whereas SENTRA relies on hardware enclaves and attests enclave state rather than the full system. Privacy-preserving attestation in multi-tenant settings has also been explored. Ferro et al. [23] use Linux IMA namespaces for isolated container verification, while Jing et al. [24] propose recursive attestation for heterogeneous clouds under coordinated infrastructure assumptions. Suriano et al. [25] study attestation of Service Function Chains in ETSI NFV using centralized orchestration. In

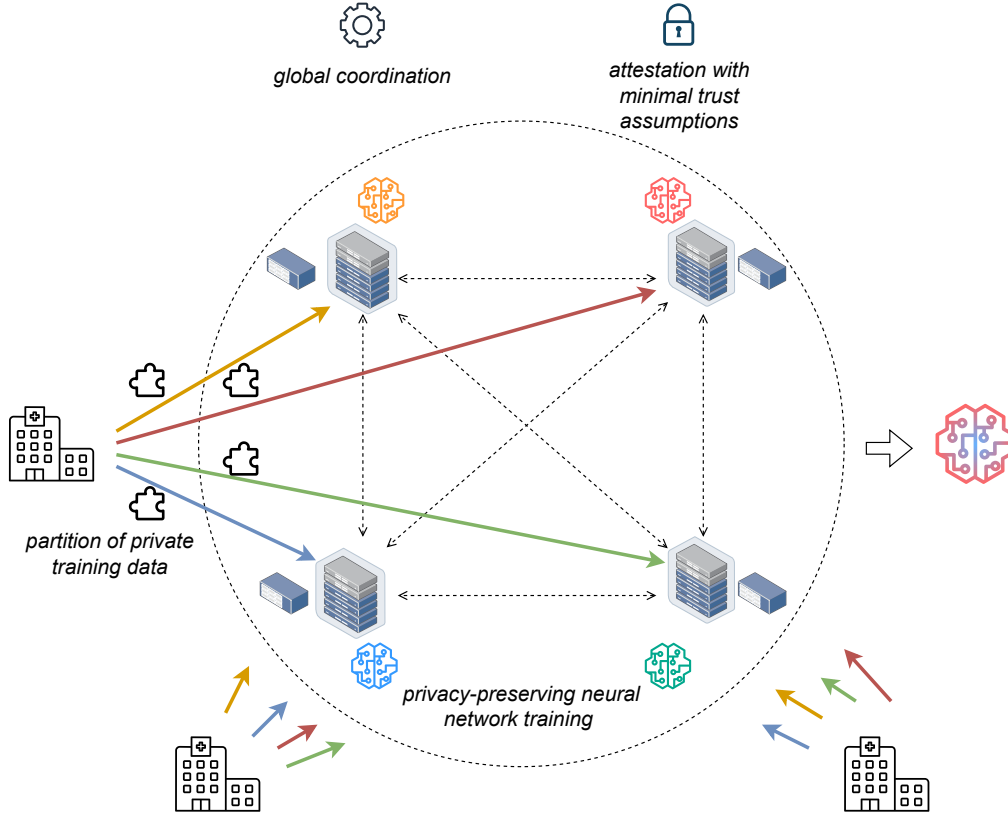


Figure 1: SENTRA in a medical setting: hospitals upload private data to distributed storage and training nodes, which train on secret shares. A central attestation service periodically re-attests nodes and coordinates storage-node selection.

contrast, SENTRA combines enclave-based isolation with decentralized attestation under stronger adversarial assumptions. A comparison of SENTRA with related frameworks is shown in Table 1.

3.2. TEE and MPC for Machine Learning

Frameworks for privacy-preserving machine learning fall into two categories: (i) MPC-based systems such as CrypTen [28], and (ii) hybrid TEE-cryptography systems including STAMP [33], FATE [34], Phantom [35], and 3LegRace [36]. Their capabilities are summarized in Table 2. CrypTen [28] provides a PyTorch-native interface [37] for secure MPC-based training and inference, but lacks attestation, rollback protection, and churn handling. Hybrid systems mainly explore efficiency-security trade-offs. STAMP [33] accelerates inference using lightweight trusted hardware, FATE [34] combines MPC, homomorphic encryption, and differential privacy for federated learning under honest-but-curious assumptions, and Phantom [35] protects model IP through TEE-GPU partitioning. 3LegRace [36] accelerates private training in TEE-GPU systems, while Chiron [29] and Ryoan [30] provide enclave-based ML privacy. Slalom [31] improves TEE-based inference us-

ing Freivalds’ algorithm [32], but does not support training resilience. Overall, MPC-first systems support secure training but lack deployment safeguards, whereas hybrid systems improve efficiency with limited resilience. SENTRA uniquely combines MPC, collective TEE attestation, versioned storage, and DPSS resharing to provide both efficiency and robustness under churn. Related TEE-GPU partitioning works [38]–[40] are outside the scope of this work because they rely on hardware accelerators.

4. SENTRA: Secure and Distributed Training System

4.1. Overview of SENTRA

Before describing the system components, we summarize the main execution stages of SENTRA, corresponding to the end-to-end protocol in Fig. 2.

a) *Collective attestation and node admission.* All storage and training nodes undergo remote attestation before admission. Each node provides an attestation report containing enclave measurements and a public-key hash bound to an epoch nonce. Only nodes with fresh, valid attestation evidence are admitted. Epoch activation occurs only

TABLE 1: Comparison of SENTRA with Related Attestation and Coordination Frameworks

Framework	Approach	Attestation	Membership Selection	Distributed	Key Advantages	Limitations
SENTRA (ours)	TEE + secret sharing + MPC with attestation-driven coordination	scalable collective attestation with epoch-based freshness guarantees	decentralized committee selection under diversity constraints	yes	unified design combining attestation, secure storage, and training with strong freshness and rollback protection	higher system complexity; relies on TEE trust assumptions and does not address microarchitectural side channels
[26]	Decentralized root key of TEE using a PUF. Distributed verification using smart contracts.	smart contracts	none	yes	trust judgements based on smart contract using pseudorandomness	
[20]	Mutual attestation within a group of enclaves without a trusted third party	mutual, no TTP	none	no	attestation without single point of failure	does not cover no membership selection
[27]	Usage of off-chain TEE nodes for offloading smart contract execution	performed by an existing member of the group	randomized, without inter-node communication after a block was added to the blockchain	yes, usage of off-chain distributed storage	off-chain execution of smart contracts, selection of TEE nodes	members only selected based on randomness
[13]	decentralized attestation using blockchain for storing historical attestation results	PRNG used for scheduling attestation, result of self-attestation is added to blockchain	no, but computation of a history-based trust score, a weighted average based on age of attestation	yes	computation of a trust score	targets IoT, self-attestation

TABLE 2: Comparison of SENTRA with related frameworks for privacy-preserving ML.

Framework	Approach	Secure Training Support	Fault Tolerance	Key Advantages / Limitations
SENTRA (ours)	TEE + Packed Shamir Secret Sharing + MPC + DPSS	Yes (full DNN training)	Yes (via Node-Join + DPSS)	End-to-end confidential training, rollback protection, and secure node churn handling. Integrates TEEs, MPC, and a distributed KVS.
CrypTen [28]	MPC-based secure computation integrated with PyTorch	Yes	No	PyTorch-native MPC with ease of integration, but lacks TEE-based security, rollback protection, and fault tolerance.
Chiron [29]	TEE (Intel SGX) + Ryoan [30] sandbox for ML-as-a-Service	Yes	No	Privacy-preserving ML using SGX enclaves and Ryoan sandbox to protect training data while keeping the model architecture confidential. Supports distributed training via parameter server, but lacks rollback protection, secret sharing, dynamic membership management, and resilience to node churn. Vulnerable to SGX side-channel and rollback limitations.
Slalom [31]	TEE + verifiable outsourcing of linear layers (Freivalds' algorithm [32]) + optional input blinding	No (inference only)	No	Efficient verifiable and private DNN inference by outsourcing linear layers from a TEE to an untrusted GPU. Achieves significant inference speedups, but does not support secure training, rollback protection, dynamic membership, or secret resharing.
STAMP [33]	MPC + Lightweight Trusted Hardware (LTH) offloading for non-linear ops	No (inference only)	No	Large inference speedups by offloading non-linear ops to LTH; supports CPU/GPU and LAN/WAN settings; malicious security with abort under honest-majority. Inference only, no rollback protection or churn handling. Makes use of LTH (e.g. TPM) instead of TEE to reduce attack surface and minimise the risk of side channel attacks.
FATE [34]	Federated learning + MPC + HE	Yes	No	Enterprise-grade federated learning platform with MPC and homomorphic encryption, but lacks rollback protection and secure resharing. No TEE usage.
Phantom [35]	TEE + obfuscation-based DNN protection	No (inference only)	No	Uses TEEs and reinforcement-learning-driven obfuscation for model confidentiality, but focuses on inference and lacks secure fault recovery.
3LegRace [36]	Hybrid TEE-GPU asymmetric partitioning	Partial	No	Splits models into TEE-protected low-rank components and GPU residuals with DP noise, improving efficiency but lacking DPSS-based resilience. Does not consider side-channel attacks.

after attestation, committee selection, and required resharing complete.

b) Secure ingestion and secret sharing. Training data and initial model parameters are encoded using Shamir or Packed Shamir Secret Sharing and distributed across attested KVS nodes. No single node can reconstruct plaintext values.

c) Retrieving shares for computation. Training nodes retrieve secret shares of minibatches and model parameters from the distributed KVS through authenticated channels bound to attestation results and the current epoch. Since all KVS shards run inside TEEs, share integrity and provenance are guaranteed.

d) Secure training execution. Forward/backward passes, gradient updates, and aggregation are performed on secret shares using communication-efficient packed MPC inside enclaves. Packed Shamir Secret Sharing amortizes computation and communication across multiple values.

e) Writing updates with freshness guarantees. Updated model shares are written back to the distributed KVS under monotonically increasing version numbers. Version-aware compare-and-set semantics enforced inside enclaves prevent stale or replayed writes, ensuring rollback protection and globally consistent model state.

f) Handling failures and dynamic membership. SENTRA tolerates node joins, departures, and failures using Dynamic Proactive Secret Sharing (DPSS), which refreshes or re-distributes shares without reconstructing plaintext. Node Join introduces newly attested nodes into the committee, while Node Dropout removes failed nodes and refreshes remaining shares. These operations are epoch-bound: after resharing, a new epoch activates, stale shares and credentials are invalidated, and only current members may access secret state. During training, SENTRA maintains the packed-computation safety condition $2(t + s - 1) < n_{\text{active}}$. If the bound is violated, SENTRA either reduces the packing factor, performs membership repair via Node Join / Node Dropout, or pauses training until safety is restored. These steps provide the high-level execution flow of SENTRA. The following subsections detail the architecture and mechanisms that implement each step.

4.2. Architecture Overview

Figure 3 illustrates SENTRA’s system architecture, which integrates a distributed, enclave-backed KVS with MPC-based training across multiple TEE nodes. At a high level, SENTRA combines three subsystems—(i) a distributed, enclave-backed key–value store (KVS), (ii) a packed-MPC execution layer, and (iii) an attestation-driven coordination service—into a unified architecture for privacy-preserving training in untrusted clouds. Each node hosts (i) a KVS shard storing Shamir or Packed Shamir shares of training data, model parameters, membership state, and DPSS resharing metadata, and (ii) an MPC engine that executes secure forward/backward passes and parameter updates. Each node in the cluster runs inside an TEE enclave and hosts both a KVS shard and an MPC execution engine. The KVS shard stores secret shares together with version metadata, while the

MPC engine performs secure computation on secret-shared inputs.

Coordinator / Coordination Service. In SENTRA, the coordinator is a *coordination service* rather than a single trusted node. It manages collective attestation, membership and epoch state, committee selection, and DPSS resharing during node joins, departures, or failures. It also enforces rollback protection through version monotonicity and verifies the packed-secret-sharing safety condition $2(t + s - 1) < n_{\text{active}}$. The coordinator operates only on protocol metadata and may be implemented as a BFT-replicated service or smart-contract-based control plane. Keys are assigned to KVS shards using HRW routing to minimize data movement during churn. A logging server maintains an attestation transparency log, while all control- and data-plane communication uses mTLS identities bound to collective attestation results. Together, these mechanisms enable scalable privacy-preserving training across mutually attested enclaves with integrity and rollback protection through versioned storage semantics.

4.3. Security Model

4.3.1. System and Assets. SENTRA enables privacy-preserving DNN training across distributed nodes running inside hardware enclaves. Each node hosts (i) a KVS shard storing Shamir or Packed Shamir shares of training data and model parameters, and (ii) an MPC engine for secure forward/backward passes and model updates on shares. A coordination service (logically centralized but BFT-tolerant) manages mini-batch execution, enforces versioned writes for rollback protection, monitors liveness, and handles membership changes (Node Join/Dropout). Remote attestation ensures that only verified TEEs participate, and all membership changes are bound to epochs. SENTRA protects: (i) training data, which remains secret-shared both in use and at rest; (ii) model parameters and intermediate states across all training iterations; (iii) protocol state, including dataset-/model versions (v_D, v_θ), membership epochs, HRW placement, and access credentials; and (iv) training correctness by preventing unauthorized tampering with computations or protocol state.

4.3.2. Security Goals. The security goals of SENTRA are as follows.

- *Data and Model Confidentiality.* No single node can reconstruct training data or model parameters; at least $t+1$ shares are required. Shares and computations remain inside TEEs; network traffic is protected by mTLS.
- *Computation Integrity and Correctness.* Only attested, authorized nodes participate; MPC engines execute the same verified protocol. Quorum and compare-and-set semantics prevent stale or unauthorized updates.
- *Freshness and Rollback Protection.* Version-aware KVS semantics and monotonically increasing model versions v_θ ensure that rollback and replay are prevented. Attestation artifacts are checked for freshness and are bound to membership epochs.

SENTRA: System- and Training-Level Protocol

Parameters: t (adversarial threshold), n (KVS / MPC nodes), s (packing factor), B (mini-batch size)

Metadata: v_D (dataset version), v_θ (model version, initially 0), e (membership epoch)

API: `put(key, value, version)`, `get(key, min_version)`

Inputs: Dataset $D = \{x_i\}_{i=1}^N$, model f_θ

Outputs: Trained parameters θ^* stored securely in the KVS

Initialize: Set $v_\theta = 0$ and start membership epoch e .

Protocol (System / Membership Epoch):

1. **Collective Attestation.** All nodes produce TEE attestation evidence bound to an epoch nonce. Only successfully attested nodes become *candidates* for the next committee.
2. **Decentralized Committee Selection.** A BFT-tolerant mechanism (or smart contract) selects the active committee and prefers reusing the previous committee when possible.
3. **Node Join / Dropout Hooks.**
 - Node-Join hook triggers collective attestation and epoch-based membership update when new nodes are admitted.
 - Node-Dropout hook triggers resharing and epoch transition when missing or failed nodes are detected.
4. **DPSS Resharing on Membership Change.** If membership differs from the previous epoch, Dynamic Proactive Secret Sharing (DPSS) refreshes shares without plaintext reconstruction; old shares become invalid.
5. **Packing-Safety Check.** Training is allowed only if

$$2(t + s - 1) < n_{\text{active}}.$$

Otherwise, SENTRA reduces s , replaces nodes, or triggers DPSS-based repair.

6. **Epoch Activation.** The new epoch $e + 1$ becomes active; credentials and shares from earlier epochs are rejected.

Protocol (Training / Algorithm Level):

7. **Secure Storage of Training Data.** Each x_i is Shamir-shared $\langle x_i \rangle = \{x_i^{(1)}, \dots, x_i^{(n)}\}$ and written into enclave-backed KVS nodes with version tag v_D .
8. **Mini-Batch Retrieval.** For $B \subset D$, nodes perform version-aware reads:

$$\text{get}(\text{key}=i, \text{min_version}=v_D) \rightarrow (x_i^{(j)}, \text{ver}_i^{(j)})$$

Training proceeds only if at least $t + 1$ fresh shares are available.

9. **Conversion to Packed Shares.** Shamir shares are converted into Packed Secret Sharing (PSS) form enabling SIMD-style secure linear algebra.
10. **Private Neural Network Training.** Forward/backward passes and secure updates are computed collaboratively using the packed-MPC runtime inside attested enclaves.
11. **Versioned Weight Update.** After each iteration, increment $v_\theta \leftarrow v_\theta + 1$ and store updated parameter shares with rollback protection.
12. **Fault Handling During Training.** If nodes fail, SENTRA pauses training and triggers Node-Join / Dropout protocols and DPSS resharing until the packing-safety bound is restored.
13. **Repeat Until Convergence.** Steps 8–12 repeat across many iterations and multiple ML epochs.

Figure 2: End-to-end SENTRA protocol combining collective attestation, epoch-based membership, DPSS resharing, and privacy-preserving training.

• Secure and Dynamic Membership.

- **Join:** Only attested nodes are admitted; DPSS resharing provides new shares without reconstructing plaintext; epoch activation updates the membership allow-list.
- **Dropout:** Training continues if (i) at least $t+1$ fresh shares per key are available and (ii) the packing-safety bound holds: $2(t+s-1) < n_{\text{active}}$, $s_{\text{max}} = \left\lfloor \frac{n_{\text{active}}}{2} - t \right\rfloor$. Otherwise, the current batch is aborted and recovery is triggered (resharing or node replacement).

• Fault Tolerance.

HRW placement minimizes data movement; DPSS resharing restores redundancy without ex-

posing plaintext; training continues in degraded mode whenever safety conditions hold.

Trust Assumptions and TCB. SENTRA relies on a set of trust assumptions defining its trusted computing base (TCB). We trust the TEE hardware and firmware to enforce enclave isolation. The coordination service is trusted only to maintain candidate-node metadata and never accesses plaintext training data or model parameters. SENTRA defends against: (i) *cloud/OS compromise*—hosts cannot access enclave memory, and collusion of up to t nodes reveals no secrets; (ii) *Byzantine participants*—malicious nodes cannot force unauthorized updates due to quorum-based

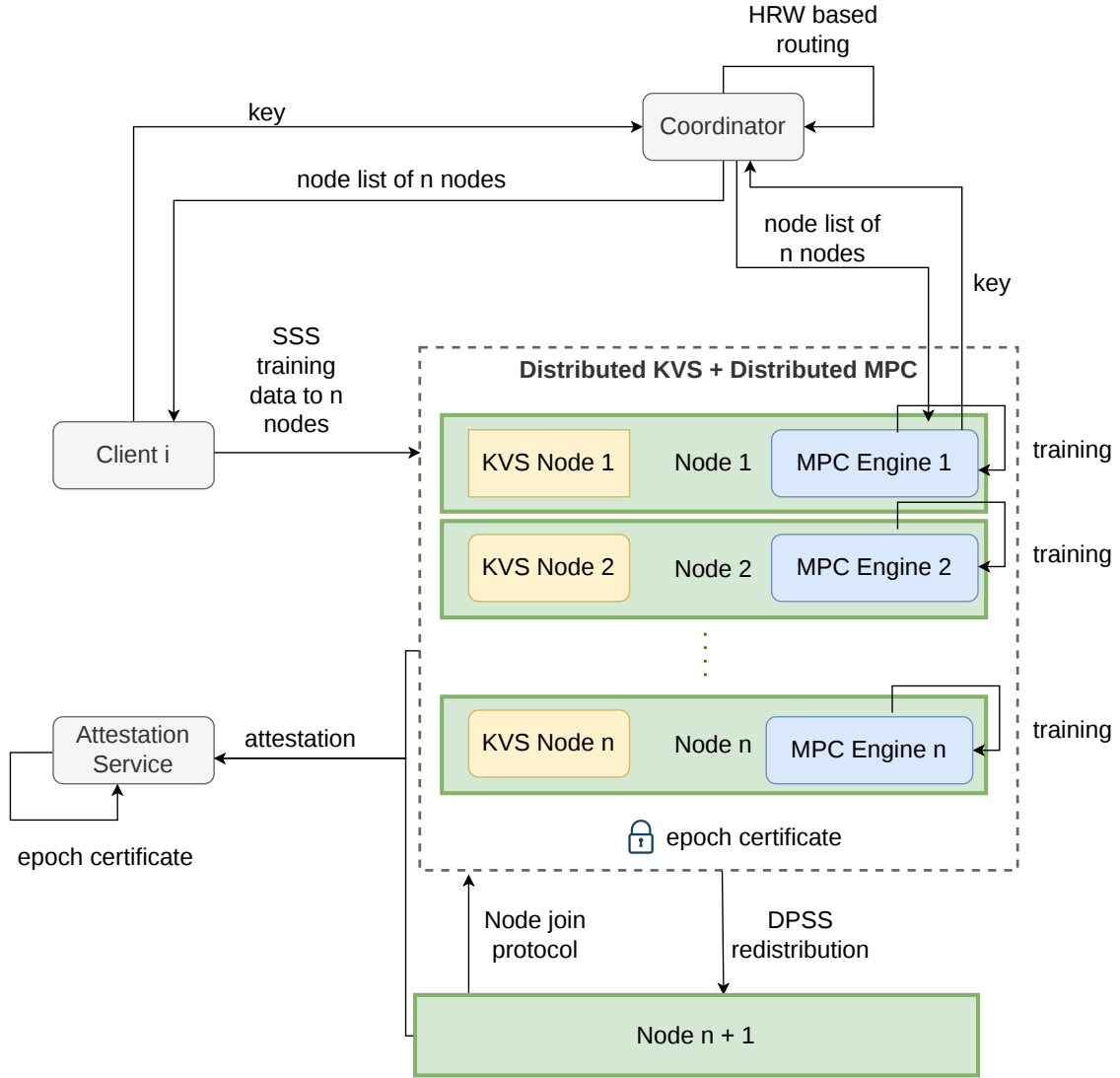


Figure 3: Architecture of SENTRA

correctness checks; (iii) *network adversaries*—mTLS ensures confidentiality and integrity, while versioning, nonces, and epoch-bound credentials prevent replay and unauthorized access; (iv) *rollback/replay attacks*—prevented through version-aware KVS semantics and coordinator consistency checks; and (v) *targeted churn attacks*—training remains safe while the packing-safety bound holds, otherwise recovery is triggered through DPSS resharing or adjustment of s . Consistent with standard confidential-computing assumptions, SENTRA does not address: (a) side- and covert-channel attacks [41] (e.g., cache, page-fault, interrupt, or single-stepping attacks). Although SENTRA reduces single-point exposure through distributed TEEs, it does not eliminate microarchitectural leakage; (b) physical hardware attacks [42]; and (c) confused-deputy [43] and denial-of-

service (DoS) attacks [44].

4.4. Core Mechanisms

We describe below the core mechanisms that collectively realize SENTRA’s end-to-end secure training pipeline.

4.4.1. Remote Attestation. Figure 4 illustrates how SENTRA establishes a fresh global trust state and updates membership at epoch boundaries. First, the coordinator announces candidate nodes for epoch $e+1$. The current committee C establishes attested TLS channels to these nodes using RA-TLS, proving that each executes approved code inside a genuine TEE. All subsequent communication uses mTLS with epoch-bound credentials. Based on attestation

results, committee C asynchronously agrees on eligible candidates. A smart contract then executes the committee selection protocol, producing committee C' for epoch $e+1$. The coordination service verifies the epoch certificate, updates the membership allow-list, and prepares share placement. If membership changes, secret shares are transferred from C to C' using DPSS [14] over attested channels without reconstructing plaintext. This process also realizes Node Join and Node Dropout during epoch transitions: newly attested nodes receive fresh shares after resharing, while departed nodes are removed through DPSS resharing. Finally, epoch $e+1$ is activated. Only nodes in the updated membership may access storage or participate in training, while stale credentials and shares are invalidated. Together, attestation, resharing, and epoch activation provide freshness, rollback resistance, and secure dynamic membership under node churn.

4.4.2. Secure Storage and Secret Sharing. After collective attestation establishes the trusted node set, SENTRA stores training data and model parameters using Shamir or Packed Shamir Secret Sharing across HRW-selected KVS shards. Since each shard runs inside an Intel TDX enclave, confidentiality is preserved both at rest and during computation. The KVS supports version-aware `put` and `get` operations. Each write carries a monotonically increasing version number— v_D for datasets and v_θ for model parameters. Enclaves enforce these versions, rejecting stale or replayed writes to prevent rollback attacks. To handle node churn, SENTRA uses DPSS to reshuffle or refresh shares when nodes join or leave, without reconstructing plaintext. After epoch rotation, obsolete shares become invalid, ensuring departed nodes retain no usable information. Together, secret sharing, attested enclaves, and versioned storage provide confidentiality and integrity for subsequent MPC computation.

4.4.3. Training Workflow with Packed MPC. Once mini-batch shares and model parameters are retrieved through versioned reads, SENTRA converts standard Shamir shares into PSS to enable SIMD-style parallelism. PSS amortizes MPC communication and computation costs by packing multiple elements into a single polynomial, accelerating the matrix-vector operations common in neural network training. All computations—forward/backward propagation, activation handling, truncation, and gradient updates—execute inside TDX enclaves using a communication-efficient packed-MPC runtime. Throughout training, nodes enforce the packed-MPC safety bound $2(t+s-1) < n_{\text{active}}$, ensuring polynomial degrees remain decodable after multiplications. If transient failures reduce the safety margin, the system lowers the packing factor s ; if the bound is violated, training pauses and recovery through DPSS resharing or node replacement is triggered. After each iteration, updated model shares are written back to the KVS with a new version number $v_\theta \leftarrow v_\theta + 1$. KVS shards enforce compare-and-set semantics for freshness, while the coordinator verifies confirmations from at least $t+1$ nodes. This process continues

until convergence, ensuring privacy, integrity, and robustness throughout training.

4.4.4. Versioning, Freshness, and Rollback Protection. SENTRA enforces freshness and rollback resistance through versioned storage semantics in the enclave-backed KVS. Dataset and model states use monotonically increasing versions, where v_D tracks dataset freshness and v_θ tracks model evolution. Reads use `get(key, min_version)` to prevent replay of stale shares. Writes apply version-aware compare-and-set semantics inside TEEs, allowing updates only when the supplied version exceeds the stored one. This prevents rollback and replay attacks from malicious providers or compromised nodes. After each update, the coordinator increments $v_\theta \leftarrow v_\theta + 1$ and commits only after acknowledgments from at least $t+1$ shards. DPSS resharing additionally binds shares to membership epochs. After an epoch transition, stale shares held by revoked or departed nodes become invalid, ensuring that only currently attested nodes contribute valid shares. Together, these mechanisms maintain globally consistent training state while preventing rollback and replay of outdated dataset or model versions.

Committee Formation. From the set of attested nodes, a committee for the next epoch is selected using a mechanism that enforces diversity constraints (e.g., hardware platform, operator, and physical host) to mitigate correlated failures and reduce the risk of concentrated compromise. Our current prototype implements a subset of this protocol, corresponding to the core steps illustrated in Fig. 7. In particular, we realize attestation-driven candidate selection and basic committee formation under simplified deployment assumptions (single hardware platform and administrative domain). A complete decentralized committee selection protocol, encompassing all steps shown in Fig. 7, is provided as a reference design in Appendix B.

4.4.5. Dynamic Membership and Fault Tolerance. SENTRA supports dynamic membership to ensure scalability and fault tolerance in untrusted cloud environments. Membership changes are handled in *epochs*: collective attestation establishes an attested membership list, DPSS resharing refreshes shares when the committee changes, and a new epoch is activated. Node joins and dropouts trigger this workflow, preserving secure training under node churn through epoch-based admission, resharing, and packing-safety enforcement.

a) Node Join. As illustrated in Fig. 4, a new node P_{new} may join the cluster only at epoch boundaries as part of the epoch-based membership update process. Before admission, the node participates in *collective attestation* and must be included in the resulting attestation certificate. Based on this certificate, the coordination service authorizes the node, updates the epoch membership allow-list, and triggers DPSS resharing to redistribute secret shares without reconstructing plaintext. The join is finalized upon epoch activation, at which point prior credentials are invalidated and P_{new} receives fresh, epoch-bound credentials. Node Join is not

Collective Attestation and Epoch-Based Membership Update

Protocol: The collective attestation protocol establishes a fresh, globally agreed trust state for all enclaves. Before epoch $e+1$, committee C of epoch e attests candidate members for committee C' via RA-TLS [45]. After consensus on candidate nodes, the committee selection protocol is executed through a smart contract. If a new committee is formed, secret shares are securely transferred to the new members using attested TLS channels. The protocol also supports **Node Join**, where newly admitted nodes receive fresh shares only after resharing and epoch activation.

1. **Announcement of Candidate Nodes.** The coordinator publishes potential candidate nodes for epoch $e+1$.
2. **RA-TLS Establishment.** Committee members of epoch e establish attested TLS channels with candidate nodes using RA-TLS [45]. Subsequent control- and data-plane communication uses mTLS with epoch-bound credentials.
3. **Consensus on Candidate Nodes.** Committee members asynchronously reach consensus on the candidate set for epoch $e+1$ based on attestation results, tolerating crashed members of committee C .
4. **Committee Selection Protocol.** A smart contract executes committee selection. The coordination service verifies the epoch certificate, authorizes admitted nodes, updates the epoch allow-list, and computes HRW-based share placement.
5. **Transfer of Secret Shares.** If committee C cannot be reused, DPSS [14] resharing transfers secret shares from C to C' through attested TLS channels without reconstructing plaintext.
6. **Epoch Activation.** After resharing, epoch $e+1$ is activated. Only nodes in the updated membership list may access secure storage or participate in training; stale credentials and shares from earlier epochs are rejected.

Figure 4: Collective Attestation and Membership Update Protocol

a standalone protocol, but is integrated into the collective attestation and epoch transition workflow.

b) Node Dropout. A node may drop out at any time due to crashes, network failures, or voluntary departure. Distributed privacy-preserving training must tolerate node failures without compromising confidentiality or correctness. As shown in Fig. 5, SENTRA combines failure detection, quorum checks, DPSS resharing, and epoch transitions to handle node churn. Failures are detected through missed heartbeats or expired leases. The system continues in degraded mode as long as at least $t+1$ fresh shares per key are available and the packing-safety bound $2(t+s-1) < n_{\text{active}}$ holds, where t is the adversarial threshold, s is the packing factor, and n_{active} is the number of currently live nodes. Training therefore continues while quorum availability and the packed-secret-sharing safety condition remain satisfied. If this condition is tight, the largest safe packing factor is $s_{\text{max}} = \lfloor \frac{n_{\text{active}}}{2} - t \rfloor$. If $s_{\text{max}} < 1$, training pauses and triggers DPSS resharing or node replacement. When a node permanently fails or when quorum or the safety bound is violated, the current mini-batch is aborted (the version v_θ is not advanced), and the coordination service either reduces s or initiates collective attestation, DPSS resharing, and new epoch activation to restore safety. The coordinator recomputes HRW-based placement and triggers DPSS resharing without reconstructing plaintext. Failed nodes are removed, replacement nodes are admitted through collective attestation, and a new epoch activates with refreshed credentials and invalidated stale shares. SENTRA further combines: (i) versioned KVS semantics for rollback protection, (ii) HRW-based reassignment with minimal data movement, (iii) DPSS resharing for confidential redistribution, (iv) PSS safety checks for MPC correctness, and (v) periodic collective attestation to exclude revoked nodes. Together, these mechanisms enable SENTRA to maintain privacy and correctness despite node churn in long-running distributed training.

Remark (Sequential Multiplications). The bound $2(t+s-$

$1) < n_{\text{active}}$ assumes degree reduction after each multiplication. With m chained multiplications before reduction, the stronger bound $2^m(t+s-1) < n_{\text{active}}$ must hold.

5. Evaluation

5.1. System Implementation

Software. SENTRA is implemented as a multi-party secure training architecture in which the data-owning client secret-shares dataset features and labels across to a committee of n nodes using PSS [7] with adversarial reconstruction threshold t and packing factor s . The implementation is available at [46]. Training is executed in batched form with inter-node coordination over a versioned KVS and synchronised barriers. Table 3 outlines the functional component breakdowns and algorithmic roles deployed within the multi-party stack.

Model and System. The model used for evaluation is a two-layer Multi-Layer Perceptron (MLP) for MNIST [49] classification and maps an infrastructure layer of dimensions: $784 \rightarrow 128 \rightarrow \text{ReLU} \rightarrow 10 \rightarrow \text{softmax}$, this model introduces $101,770 \approx 0.1\text{M}$ parameters. To preserve confidentiality, the nodes execute training without plaintext visibility into training features and labels. Our SENTRA framework was developed using Python 3.10, TensorFlow 2.11 (without GPU acceleration) and NumPy 1.26.4. SENTRA supports execution across two runtime paradigms deployed via Docker containers which are executed within a virtual machine based on Linux QEMU/KVM *a) Non-SGX Mode:* A containerised software-only framework, *b) SGX Mode:* A hardware-enclave isolated framework managed by the Occlum LibOS [50] abstraction layer. The underlying hardware used for benchmarking was an INTEL® XEON® GOLD 6526Y processor. The virtual machine was configured using 16 virtual cores and 120 GB of main memory, whereas the maximum enclave memory was configured with

Node Dropout Protocol

Protocol: To tolerate transient or permanent node loss during training, SENTRA combines versioned KVS semantics, HRW key placement, quorum reads/writes, DPSS resharing, and epoch-based membership control without reconstructing plaintext. This preserves confidentiality, rollback protection, and packed-secret-sharing correctness.

Notation: n_{active} denotes the number of live nodes returning *fresh* shares for *all keys touched* in the current mini-batch.

1. **Detection & Grace Window.** Nodes missing heartbeats or leases are marked *Suspect* for a grace period Δ_{detect} . Reads continue from remaining HRW-assigned nodes. If at least $t+1$ fresh shares per key are available, training proceeds in degraded mode; otherwise the node is marked *Failed*.
2. **Safe Degraded Progress.** While $n_{\text{active}} \geq t+1$ and

$$2(t + s - 1) < n_{\text{active}},$$

iterations continue using quorum reads/writes with compare-and-set semantics. Model version v_θ advances only after $\geq t+1$ confirmations; stale writes are rejected.

3. **PSS Safety Check.** Let s be the packing factor. Since packed polynomials have degree $d = t + s - 1$ and multiplications double degree, verify before each mini-batch:

$$2(t + s - 1) < n_{\text{active}}.$$

Equality is treated as unsafe. The maximum safe packing factor is:

$$s_{\text{max}} = \left\lfloor \frac{n_{\text{active}}}{2} - t \right\rfloor.$$

Enforce $s \leftarrow \min(s, s_{\text{max}})$. If $s_{\text{max}} < 1$, pause training and trigger recovery. If the bound is violated, either:

- reduce s (optionally per-layer), or
- repair via DPSS resharing or node replacement.

For m chained multiplications before degree reduction, use:

$$2^m(t + s - 1) < n_{\text{active}}.$$

4. **Scale-In Recovery (Permanent Dropout).** For a failed node, the coordination service creates a new membership epoch, recomputes HRW placement so only affected keys move, and performs DPSS resharing:

$$C' = C \setminus \{P_{\text{failed}}\},$$

restoring redundancy without plaintext reconstruction.

5. **Scale-Out with Replacement.** If a replacement node exists, it is admitted during the next epoch transition via collective attestation and membership update (cf. Fig. 4). DPSS resharing proceeds as:

$$C'' = (C \setminus \{P_{\text{failed}}\}) \cup \{P_{\text{new}}\}.$$

6. **Epoch Activation & Access Control.** After resharing, a new epoch activates; allow-lists and JWT credentials rotate automatically, and stale credentials expire. Only authorized nodes may access versioned shares.
7. **Mini-Batch Boundary Semantics.** If any touched key cannot gather $\geq t+1$ fresh shares, abort the mini-batch, do not advance v_θ , and trigger recovery. Commit occurs only after quorum confirmation of the new v_θ .
8. **Security & Freshness Guarantees.** DPSS resharing never reconstructs plaintext, and revoked nodes retain only obsolete shares after epoch rotation. Versioned put/get with `min_version` prevents rollback, while HRW minimizes data movement.

Figure 5: Node Dropout Protocol

TABLE 3: Functional components

Component	Role
Linear layers	Beaver-triple-based multiplication [47] on secret-shared tensors
ReLU	Secure comparison
Softmax / loss	Fixed-point secure softmax; Padé(2,2) exponential approximation [48]
Gradients	Protocol-compliant secure approximation
Arithmetic	61-bit prime field, scale factor 2^{16}

32 GB. Note that the available main memory is much larger than needed for running the evaluation avoiding swapping. The baseline training parameters utilised throughout the core

evaluation process were selected via an initial preliminary validation sweep to balance algorithmic convergence with fixed-point execution stability. The evaluation workload is

governed by the specialised configuration profile detailed in Table 4, unless specified otherwise.

TABLE 4: Configuration parameters used during evaluation

Setting	Value
Nodes	3
Training Samples	10,000
Test Samples	100
Batch Size	64
Epochs	10
Learning Rate	0.002
Seed	2026
Loss Mode	Softmax (temperature 2.0)

5.2. TEE Deployment

The committee selection protocol assumes diversity across hardware platforms, operators, and physical machines. Our implementation evaluates only Intel SGX on our infrastructure and does not extract machine identifiers from attestation reports. This demonstrates feasibility on Intel SGX; support for other TEEs such as AMD SEV-SNP is beyond the scope of this work. Operator and machine identifiers are assumed to be provided by an external manifest layer. SENTRA secures both storage and computation using TEEs. To our knowledge, it is the first distributed training system combining collective TEE attestation, epoch-bound membership, and DPSS resharing to tolerate node churn under packed-MPC constraints. Table 5 compares SENTRA with CrypTen and TruShare. Unlike these systems, SENTRA integrates TEE isolation, packed MPC, DPSS resharing, and versioned storage to provide rollback protection, dynamic membership, and resilient privacy-preserving distributed training. Compared to CrypTen [28], which assumes static MPC participants, SENTRA adds protections against rollback attacks, stale membership reuse, and node churn, enabling resilient cloud-native confidential MPC training. Performance comparisons, including SENTRA without confidential computing, are shown in Table 6.

5.3. Benchmarking

As detailed in Table 6, SENTRA Non-SGX Mode achieves optimal runtime efficiency across the benchmark suite, completing 10-epoch training task in 3.15 hours, hence outperforming CrypTen by $1.29\times$. Using SGX Mode introduces a distinct performance tax, extending the secure training period to 4.40 hours and peaking memory usage to 4.58GB. However, this minor 8.3% execution premium over CrypTen achieves a fundamentally superior threat profile. CrypTen assumes a trusted host infrastructure, whereas SENTRA’s SGX deployment strictly isolates data structures to prevent malicious host-operator snooping. Furthermore, when unexpected node dropouts occur mid-training, SENTRA demonstrates robust resilience, triggering Lagrange-based re-sharing [52] to resume training within an 8 – 9s window. The chart, illustrated in Fig. 6, provides a granular

overview of the computational overhead of the sub-protocols on the Prover instance, directly contrasting Non-SGX and SGX configurations. Both deployments is mostly driven by the Opened divide & reshare primitive. However, the transition of identical training logic into an isolated hardware enclave, surfaces a $3.6\times$ execution penalty.

Acknowledgments

The authors affiliated with the Barkhausen Institut were supported by the German Federal Ministry of Research, Technology, and Space through the SEMECO-Q1 project (Grant No. 03ZU2210AA) and the 6G-CampuSens project (Grant No. 16KISK205). Additional financial support was provided through funds allocated by the Saxon State Parliament in Germany.

References

- [1] M. Sabt, M. Achemlal, and A. Bouabdallah, “Trusted execution environment: What it is, and what it is not,” in *2015 IEEE Trust-com/BigDataSE/ISPA*, vol. 1. IEEE, 2015, pp. 57–64.
- [2] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, “Remote attestation procedures (RATS) architecture,” (*No Title*), 2023.
- [3] F. McKeen, I. Alexandrovich, A. Berenzon, C. V. Rozas, H. Shafi, V. Shanbhogue, and U. R. Savagaonkar, “Innovative instructions and software model for isolated execution,” *Hasp@ isca*, vol. 10, no. 1, 2013.
- [4] V. Costan and S. Devadas, “Intel SGX explained,” *Cryptology ePrint Archive*, 2016.
- [5] J. Van Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx, “Foreshadow: Extracting the keys to the Intel SGX kingdom with transient Out-of-Order execution,” in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 991–1008.
- [6] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [7] M. Franklin and M. Yung, “Communication complexity of secure computation,” in *Proceedings of the twenty-fourth annual ACM symposium on Theory of computing*, 1992, pp. 699–710.
- [8] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, “Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy,” in *International conference on machine learning*. PMLR, 2016, pp. 201–210.
- [9] S. Wagh, D. Gupta, and N. Chandran, “SecureNN: 3-party secure computation for neural network training,” *Proceedings on Privacy Enhancing Technologies*, 2019.
- [10] P. Mohassel and Y. Zhang, “Secureml: A system for scalable privacy-preserving machine learning,” in *2017 IEEE symposium on security and privacy (SP)*. IEEE, 2017, pp. 19–38.
- [11] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, “Dynamo: Amazon’s highly available key-value store,” *ACM SIGOPS operating systems review*, vol. 41, no. 6, pp. 205–220, 2007.
- [12] X. Zhang, K. Qin, S. Qu, T. Wang, C. Zhang, and D. Gu, “Teamwork makes tee work: open and resilient remote attestation on decentralized trust,” *IEEE Transactions on Dependable and Secure Computing*, 2025.
- [13] S. F. J. J. Ankergr ard, E. Dushku, and N. Dragoni, “PERMANENT: Publicly Verifiable Remote Attestation for Internet of Things Through Blockchain,” in *International Symposium on Foundations and Practice of Security*. Springer, 2021, pp. 218–234.

TABLE 5: High-level comparison of SENTRA with representative baselines.

System	Security Model	Lifecycle Coverage	Performance
CrypTen [28]	Pure MPC	Training only	Moderate
TruShare [51]	TEE + SSS	Storage focus	High overhead
SENTRA	TEE + Packed MPC + DPSS + Versioned KVS	Full lifecycle	Balanced

TABLE 6: Benchmark Comparison: SENTRA vs CrypTen [28]

Metric (Measured)	CrypTen [28]	SENTRA	
		SGX	Non-SGX
Training (h)	4.06	4.40	3.15
Training Throughput (samples/sec)	6.83	6.32	8.89
Per-Iteration Latency (ms)	9.32	10.18	7.19
Client Cold-Start (s)	-	251	1.05
Client PSS Distribution (s)	-	220	129
Communication Volume / Iteration (MB)	-	24.91	24.00
Peak Memory Usage (GB)	-	4.58	3.26
Failure Recovery Time (sec)	-	8.89	8.09
Versioning Overhead (%)	-	0.05	0.05
Final Accuracy (%)	83	76	76

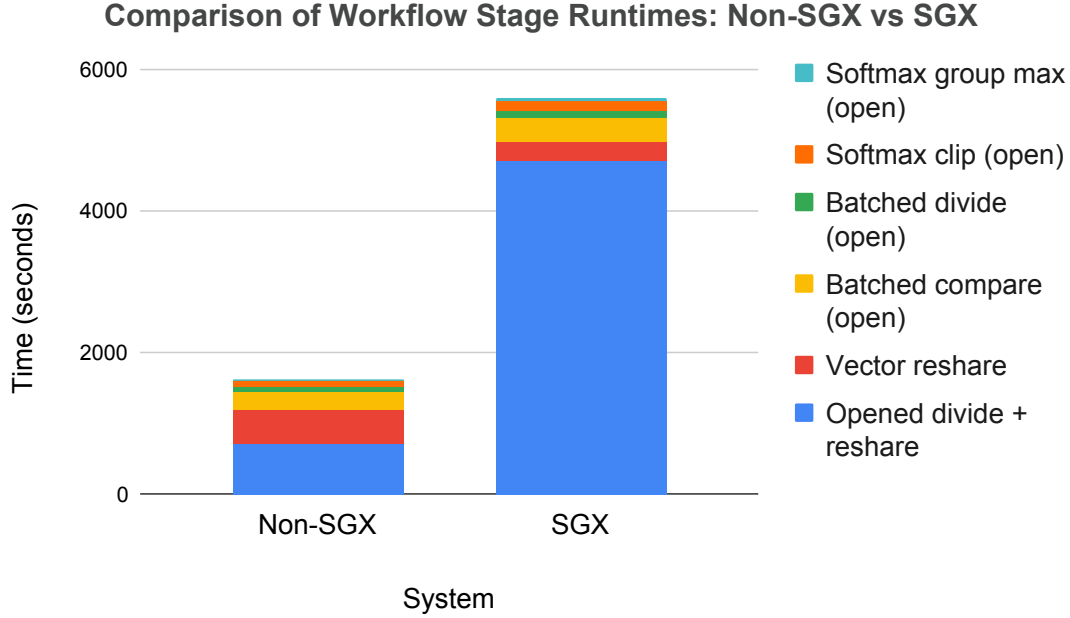


Figure 6: Prover Sub-Protocol Overhead Comparison

- [14] T. Yurek, Z. Xiang, Y. Xia, and A. Miller, “Long live the honey badger: Robust asynchronous DPSS and its applications,” in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 5413–5430.
- [15] L. Coppolino, S. D’Antonio, G. Mazzeo, and L. Romano, “An experimental evaluation of TEE technology: Benchmarking transparent approaches based on SGX, SEV, and TDX,” *Computers & Security*, vol. 154, p. 104457, 2025. [Online]. Available: <https://doi.org/10.1016/j.cose.2025.104457>
- [16] D. Giantsidi, M. Bailleu, N. Crooks, and P. Bhatotia, “Treaty: Secure distributed transactions,” in *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2022, pp. 14–27.
- [17] D. G. Thaler and C. V. Ravishanker, “Using name-based mappings to increase hit rates,” *IEEE/ACM Transactions on networking*, vol. 6, no. 1, pp. 1–14, 2002.
- [18] N. Asokan, F. Brasser, A. Ibrahim, A.-R. Sadeghi, M. Schunter, G. Tsudik, and C. Wachsmann, “Seda: Scalable embedded device attestation,” in *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, 2015, pp. 964–975.
- [19] B. Kuang, A. Fu, S. Yu, G. Yang, M. Su, and Y. Zhang, “ESDRA: An efficient and secure distributed remote attestation scheme for IoT swarms,” *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8372–8383, 2019.
- [20] G. Chen and Y. Zhang, “MAGE: Mutual attestation for a group of enclaves without trusted third parties,” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 4095–4110.

- [21] S. Ott, M. Kamhuber, J. Pecholt, and S. Wessel, "Universal remote attestation for cloud and edge platforms," in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, 2023, pp. 1–11.
- [22] M. Eckel, D. R. George, B. Grohmann, and C. Krauß, "Remote attestation with constrained disclosure," in *Proceedings of the 39th Annual Computer Security Applications Conference*, 2023, pp. 718–731.
- [23] L. Ferro, E. Bravi, S. Sisinni, and A. Lioy, "Privacy-Preserving Container Attestation," *Journal of Network and Systems Management*, vol. 34, no. 1, p. 5, 2026.
- [24] Y. Jing, M. Steiner, A. Vahldiek-Oberwagner, M. Vij, and L. Vilanova, "Using Recursive Attestation to Scale Trust in Modern Heterogeneous Cloud Architectures," in *Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems*, 2025, pp. 185–193.
- [25] A. Suriano, D. Striccoli, G. Piro, R. Bolla, and G. Boggia, "Attestation of trusted and reliable service function chains in the ETSI-NFV framework," in *2020 6th IEEE Conference on Network Softwarization (NetSoft)*. IEEE, 2020, pp. 479–486.
- [26] X. Zhang, K. Qin, S. Qu, T. Wang, C. Zhang, and D. Gu, "Teamwork makes tee work: open and resilient remote attestation on decentralized trust," *IEEE Transactions on Dependable and Secure Computing*, 2025.
- [27] K. Zhang and A. Martin, "RaceTEE: A Practical Privacy-Preserving Off-Chain Smart Contract Execution Architecture," *arXiv preprint arXiv:2503.09317*, 2025.
- [28] B. Knott, S. Venkataraman, A. Hannun, S. Sengupta, M. Ibrahim, and L. van der Maaten, "Crypten: Secure multi-party computation meets machine learning," *Advances in Neural Information Processing Systems*, vol. 34, pp. 4961–4973, 2021.
- [29] T. Hunt, C. Song, R. Shokri, V. Shmatikov, and E. Witchel, "Chiron: Privacy-preserving machine learning as a service," *arXiv preprint arXiv:1803.05961*, 2018.
- [30] T. Hunt, Z. Zhu, Y. Xu, S. Peter, and E. Witchel, "Ryoan: A distributed sandbox for untrusted computation on secret data," *ACM Transactions on Computer Systems (TOCS)*, vol. 35, no. 4, pp. 1–32, 2018.
- [31] F. Tramèr and D. Boneh, "Slalom: Fast, verifiable and private execution of neural networks in trusted hardware," *arXiv preprint arXiv:1806.03287*, 2018.
- [32] R. Freivalds, "Probabilistic machines can use less running time," in *IFIP congress*, vol. 839, 1977, p. 842.
- [33] P. Huang, T. Hoang, Y. Li, E. Shi, and G. E. Suh, "Efficient Privacy-Preserving Machine Learning with Lightweight Trusted Hardware," *Proceedings on Privacy Enhancing Technologies*, vol. 4, pp. 327–348, 2024.
- [34] Y. Liu, T. Fan, T. Chen, Q. Xu, and Q. Yang, "Fate: An industrial grade platform for collaborative learning with data protection," *Journal of Machine Learning Research*, vol. 22, no. 226, pp. 1–6, 2021.
- [35] J. Bai, M. H. I. Chowdhury, J. Li, F. Yao, C. Chakrabarti, and D. Fan, "Phantom: Privacy-Preserving Deep Neural Network Model Obfuscation in Heterogeneous TEE and GPU System," 34th USENIX Security Symposium, 2025.
- [36] Y. Niu, R. E. Ali, and S. Avestimehr, "3LegRace: Privacy-Preserving DNN Training over TEEs and GPUs," *Proceedings on Privacy Enhancing Technologies*, vol. 4, pp. 183–203, 2022.
- [37] A. Paszke, S. Gross, S. Chintala, and G. Chanan, "Pytorch: Tensors and dynamic neural networks in Python with strong GPU acceleration," *PyTorch: Tensors and dynamic neural networks in Python with strong GPU acceleration*, vol. 6, no. 3, p. 67, 2017.
- [38] H. Hashemi, Y. Wang, and M. Annavaram, "DarKnight: An accelerated framework for privacy and integrity preserving deep learning using trusted hardware," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 212–224.
- [39] T. Zhou, Y. Luo, S. Ren, and X. Xu, "NnSplitter: an active defense solution for dnn model via automated weight obfuscation," in *International Conference on Machine Learning*. PMLR, 2023, pp. 42 614–42 624.
- [40] D. Li, Z. Zhang, M. Yao, Y. Cai, Y. Guo, and X. Chen, "Teeslice: Protecting sensitive neural network models in trusted execution environments when attackers have pre-trained models," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 6, pp. 1–49, 2025.
- [41] D. Lantz, "Detection of side-channel attacks targeting Intel SGX," 2021.
- [42] Z. Gu, E. Valdez, S. Ahmed, J. J. Stephen, M. Le, H. Jamjoom, S. Zhao, and Z. Lin, "NVIDIA GPU Confidential Computing Demystified," *arXiv preprint arXiv:2507.02770*, 2025.
- [43] A. Machiry, E. Gustafson, C. Spensky, C. Salls, N. Stephens, R. Wang, A. Bianchi, Y. R. Choe, C. Kruegel, and G. Vigna, "BOOMERANG: Exploiting the Semantic Gap in Trusted Execution Environments," in *NDSS*, 2017, p. 35.
- [44] F. Mo, Z. Tarkhani, and H. Haddadi, "Machine learning with confidential computing: A systematization of knowledge," *ACM computing surveys*, vol. 56, no. 11, pp. 1–40, 2024.
- [45] C. Weinhold, M. U. Sardar, I. Mihalcea, Y. Deshpande, H. Tschofenig, Y. Sheffer, T. Fossati, and M. Roitzsch, "Separate but Together: Integrating Remote Attestation into TLS," in *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, 2025, pp. 1319–1326.
- [46] "SENTRA: Secure ENclave-based TRaining Architecture," <https://github.com/Barkhausen-Institut/SENTRA/>, 2026.
- [47] D. Beaver, "Efficient Multiparty Protocols Using Circuit Randomization," in *Advances in Cryptology — CRYPTO '91*, ser. Lecture Notes in Computer Science, J. Feigenbaum, Ed., vol. 576. Springer, 1992, pp. 420–432.
- [48] G. A. B. Jr. and P. Graves-Morris, *Padé Approximants*, 2nd ed. Cambridge University Press, 1996.
- [49] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [50] Y. Shen, H. Tian, Y. Chen, K. Chen, R. Wang, Y. Xu, Y. Xia, and S. Yan, "Occlum: Secure and efficient multitasking inside a single enclave of Intel SGX," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 955–970.
- [51] A. Ait Messaoud, S. B. Mokhtar, and A. Simonet-Boulogne, "TruShare: Confidential Key-Value Store for Untrusted Environments," in *20th European Dependable Computing Conference (EDCC'25)*, 2025.
- [52] A. Herzberg, S. Jarecki, H. Krawczyk, and M. Yung, "Proactive secret sharing or: How to cope with perpetual leakage," in *annual international cryptology conference*. Springer, 1995, pp. 339–352.
- [53] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, "Remote ATtestation procedureS (RATS) Architecture," RFC 9334, Jan. 2023. [Online]. Available: <https://www.rfc-editor.org/info/rfc9334>
- [54] H. Robbins and S. Monroe, "A stochastic approximation method," *The annals of mathematical statistics*, pp. 400–407, 1951.
- [55] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *Proceedings of COMPSTAT'2010: 19th International Conference on Computational Statistics Paris France, August 22-27, 2010 Keynote, Invited and Contributed Papers*. Springer, 2010, pp. 177–186.
- [56] T. ElGamal, "A public key cryptosystem and a signature scheme based on discrete logarithms," *IEEE transactions on information theory*, vol. 31, no. 4, pp. 469–472, 1985.

Appendix A. Deferred Preliminaries

Definition 7 (Confidential Computing). Confidential Computing provides protection of data in use by using hardware-based Trusted Execution Environments (TEEs). Remote attestation is one technique used in confidential computing to verify the integrity and authenticity of the execution environment before handling sensitive data. The Remote Attestation procedureS (RATS) Architecture [53] describes the following parties: i) *Attester*: produces believable information about itself called the *Evidence*, ii) *Relying Party*: decides whether or not to consider the Attester trustworthy iii) *Verifier*: which appraises the evidence and, based on that, creates Attestation Results for the Relying Party.

A.1. Training Process

When training a ML model such as logistic or linear regression, the model architecture—typically a weight vector $\mathbf{w}$ and optional bias term b —is defined in advance. The goal of training is to iteratively adjust these parameters to minimize a predefined loss function $\mathcal{L}$, which measures the discrepancy between the model’s predictions and the true labels. Each training sample consists of an input feature vector $\mathbf{x}_i \in \mathbb{R}^d$ and an associated label y_i . Among the various optimization techniques, Stochastic Gradient Descent (SGD) [54], [55] is one of the most widely used methods for training such models. Unlike full-batch gradient descent, which computes gradients over the entire dataset, mini-batch SGD estimates gradients using small, randomly sampled subsets (mini-batches) at each iteration. This approach achieves a good balance between computational efficiency and gradient stability: it enables faster updates than full-batch methods while being less noisy than sample-wise SGD.

Epochs and Mini-Batches. Training proceeds in multiple *epochs*. At the start of each epoch, the dataset D is shuffled and partitioned into K mini-batches $\mathcal{B}_1, \dots, \mathcal{B}_K$, each containing N samples. An epoch thus consists of K iterations, with each iteration processing a different mini-batch.

In the k -th iteration, let $\mathcal{B}_k = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ denote the current mini-batch. The SGD update consists of the following steps:

- 1) *Prediction*: For each sample in the mini-batch, the model computes a predicted output $\hat{y}_i$ based on the current weights:

$$\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b),$$

where $\sigma(\cdot)$ is an activation function (e.g., the sigmoid function for logistic regression or the identity function for linear regression).

- 2) *Gradient Computation*: The gradient of the loss function with respect to the model parameters is computed over the mini-batch:

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \mathbf{x}_i, \quad \nabla_b \mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i).$$

These gradients represent how the weights and bias should be adjusted to reduce the loss.

- 3) *Parameter Update*: Finally, the model parameters are updated using the gradient descent rule:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \nabla_{\mathbf{w}} \mathcal{L}, \quad b \leftarrow b - \alpha \cdot \nabla_b \mathcal{L},$$

where $\alpha > 0$ is the learning rate. This process is repeated for each mini-batch and epoch until convergence.

Appendix B. Distributed Committee Selection Protocol

The committee selection protocol, presented in Fig. 7, determines the set of active training nodes for the next epoch. Its design ensures that committee formation is transparent, decentralized, and tightly coupled to verifiable attestation evidence. Following the trust-history model of PERMANENT [13], each node maintains a reputation-like trust score reflecting its historical attestation behavior. The priority ordering relies on trust-weighted pseudorandomness, conceptually inspired by JANUS [26], which uses smart-contract-mediated, publicly auditable trust judgments. SENTRA does not adopt JANUS’s PUF-based mutual attestation; the influence is conceptual: trust data and randomness are anchored on-chain, while attestation itself remains the standard TDX DCAP mechanism described in Section 4.4.1. Node selection operates only on nodes with a valid attestation in the current or previous epoch.

At the beginning of epoch e , the committee selection contract constructs the eligible set $\mathbf{E}$. A node i is eligible if: (i) $\text{status}_i = \text{CANDIDATE}$; (ii) it is not revoked; (iii) $\text{trustScore}_i \geq \text{MIN_TRUST}$; and (iv) $(t_{\text{now}} - \text{lastAttTime}_i) \leq \text{MAX_ATTEST_AGE}$. The protocol aborts if $|\mathbf{E}| < N_{\text{target}}$, since meeting the diversity constraints would otherwise be impossible.

To minimize unnecessary resharing and preserve training efficiency, the protocol first attempts to reuse the committee of epoch e . If all members of $\mathbf{S}(e)$ remain eligible (i.e., $\mathbf{S}(e) \subseteq \mathbf{E}$) and the set $\mathbf{S}(e)$ satisfies the diversity constraints (MAX_HW_FRAC , MAX_OP_FRAC , MAX_PM_FRAC), then the committee is reused by setting $\mathbf{S}(e+1) \leftarrow \mathbf{S}(e)$, and no reselection is performed. Only if reuse fails does the protocol proceed with score computation and diversity-aware selection to construct a new committee.

If committee reuse is not possible, a trust-weighted pseudorandom score is computed for each eligible node i :

$$\text{score}_i = H(r_e \parallel \text{NodeID}_i) \cdot \text{normalize}(\text{trustScore}_i).$$

Here the value $H(r_e \parallel \text{NodeID}_i)$ is interpreted as a pseudorandom value in $(0, 1]$. Multiplying this uniform pseudorandom value by the normalized trust score w_i produces a

Distributed Committee Selection Protocol

Parameters: N_{target} , MIN_TRUST, MAX_ATTEST_AGE, MAX_HW_FRAC, MAX_OP_FRAC, MAX_PM_FRAC, per-epoch randomness r_e .

Inputs: On-chain records containing $(\text{NodeID}_i, \text{operatorID}_i, \text{hwPlatformID}_i, \text{physicalMachineID}_i, \text{measurementHash}_i, \text{lastAttTime}_i, \text{trustScore}_i, \text{status}_i)$.

Outputs: Committee $\mathbf{S}(e+1)$ for epoch $e+1$.

Initialize: $\mathbf{S}(e+1) \leftarrow \emptyset$; $\text{countHW}[\cdot] \leftarrow 0$; $\text{countOP}[\cdot] \leftarrow 0$; $\text{countPM}[\cdot] \leftarrow 0$.

Protocol:

1. **Eligibility Filtering.** Construct:

$$\mathbf{E} = \{i \mid \text{status}_i = \text{CANDIDATE}, \neg \text{revoked}_i, \text{trustScore}_i \geq \text{MIN_TRUST}, (t_{\text{now}} - \text{lastAttTime}_i) \leq \text{MAX_ATTEST_AGE}\}.$$

Abort if $|\mathbf{E}| < N_{\text{target}}$.

2. **Stability Preference (Committee Reuse).** Let $\mathbf{S}(e)$ denote the active committee of epoch e .

If all nodes in $\mathbf{S}(e)$ remain eligible (i.e., $\mathbf{S}(e) \subseteq \mathbf{E}$) and the set $\mathbf{S}(e)$ satisfies the diversity constraints (MAX_HW_FRAC, MAX_OP_FRAC, MAX_PM_FRAC), then set

$$\mathbf{S}(e+1) \leftarrow \mathbf{S}(e)$$

and proceed directly to Step 6.

Otherwise, initialize $\mathbf{S}(e+1)$ with all nodes in $\mathbf{S}(e) \cap \mathbf{E}$ whose inclusion does not violate the diversity constraints. Update the corresponding diversity counters accordingly, and continue with Step 3 to select additional nodes until $|\mathbf{S}(e+1)| = N_{\text{target}}$.

3. **Compute Scores.** For each $i \in \mathbf{E}$:

$$\text{score}_i = H(r_e \parallel \text{NodeID}_i) \cdot \text{normalize}(\text{trustScore}_i).$$

(In our current implementation, the trust-score weighting is not applied.)

4. **Sort Candidates.** Sort $\mathbf{E}$ by descending score_i to obtain $\mathbf{E}_{\text{sorted}}$. (In our prototype, ordering is based on a pre-assigned random value per node.)

5. **Greedy Diversity-Aware Selection.** For $k = 1$ to $|\mathbf{E}_{\text{sorted}}|$:

- Let $i = P_{(k)}$ with classes: $\text{hwPlatformID}_i, \text{operatorID}_i, \text{physicalMachineID}_i$.
- Compute tentative fractions:

$$\begin{aligned} f_{\text{HW}}^{\text{new}} &= \frac{\text{countHW}[\text{hwPlatformID}_i] + 1}{N_{\text{target}}}, \\ f_{\text{OP}}^{\text{new}} &= \frac{\text{countOP}[\text{operatorID}_i] + 1}{N_{\text{target}}}, \\ f_{\text{PM}}^{\text{new}} &= \frac{\text{countPM}[\text{physicalMachineID}_i] + 1}{N_{\text{target}}}. \end{aligned}$$

- If all constraints hold:

$$f_{\text{HW}}^{\text{new}} \leq \text{MAX_HW_FRAC}, \quad f_{\text{OP}}^{\text{new}} \leq \text{MAX_OP_FRAC}, \quad f_{\text{PM}}^{\text{new}} \leq \text{MAX_PM_FRAC},$$

then:

$$\begin{aligned} \mathbf{S}(e+1) &\leftarrow \mathbf{S}(e+1) \cup \{i\}, \\ \text{countHW}[\text{hwPlatformID}_i] &\leftarrow \text{countHW}[\text{hwPlatformID}_i] + 1, \\ \text{countOP}[\text{operatorID}_i] &\leftarrow \text{countOP}[\text{operatorID}_i] + 1, \\ \text{countPM}[\text{physicalMachineID}_i] &\leftarrow \text{countPM}[\text{physicalMachineID}_i] + 1. \end{aligned}$$

Set $\text{status}_i \leftarrow \text{ACTIVE}$.

- Stop if $|\mathbf{S}(e+1)| = N_{\text{target}}$.

6. **Finalize.** Emit $\text{NewCommittee}(e+1, \mathbf{S}(e+1))$.

Figure 7: Distributed committee selection protocol for epoch $e+1$. The current prototype implements a subset of this protocol under simplified deployment assumptions.

weighted random score $U_i w_i$, which is not uniform unless $w_i = 1$. Its expected value becomes $\mathbb{E}[U_i w_i] = \frac{1}{2} w_i$, giving higher-trust nodes a statistically higher score while preserv-

ing unpredictability. This behavior differs from multiplicative masking in algebraic groups (e.g., ElGamal [56]), where uniformity is preserved. In our setting, the intentional non-

TABLE 7: Parameters and notation used in the distributed committee selection protocol.

Symbol / Notation	Meaning
$\mathbf{E}$	Set of all eligible candidate nodes for selection.
$\mathbf{E}_{\text{sorted}}$	Ordered list of eligible nodes sorted by descending score.
$\mathbf{S}(e + 1)$	Selected committee for epoch $e + 1$.
N_{target}	Desired committee size for the next epoch.
MIN_TRUST	Minimum trust score required for eligibility.
MAX_ATTEST_AGE	Maximum allowed age of the last valid attestation.
MAX_HW_FRAC	Max. fraction of selected nodes sharing a hardware platform.
MAX_OP_FRAC	Max. fraction of selected nodes sharing an operator.
MAX_PM_FRAC	Max. fraction of selected nodes sharing a physical host.
r_e	Epoch randomness used for randomized selection.
NodeID_i	Unique identifier of node i .
trustScore_i	Trust/reputation score for node i .
lastAttTime_i	Timestamp of last successful attestation.
operatorID_i	Operator / cloud vendor of node i .
hwPlatformID_i	Hardware/TEE platform identifier.
$\text{physicalMachineID}_i$	Identifier of the physical host.
measurementHash_i	TEE measurement hash (e.g., MRTD, MRCONFIGID).
status_i	Node state: NONE, CANDIDATE, ACTIVE, BANNED.
$H(\cdot)$	Cryptographic hash for randomness.
score_i	Trust-weighted pseudorandom ordering score.
t_{now}	Current blockchain timestamp.

uniformity enables trust-aware yet non-manipulable committee ordering.

After sorting $\mathbf{E}$ by descending score_i , the protocol applies a greedy, diversity-aware selection pass to construct $\mathbf{S}(e + 1)$. Before including a candidate, the contract checks that adding it does not exceed any per-class capacity bound (hardware platform, operator, physical host). Using N_{target} as the denominator enforces fixed limits such as $\lfloor \text{MAX_OP_FRAC} \cdot N_{\text{target}} \rfloor$, preventing dominance by a single operator or platform.

When all constraints are satisfied, the node is added to the committee and counters are updated. The process stops once $|\mathbf{S}(e + 1)| = N_{\text{target}}$ or all candidates have been processed. A `NewCommittee` event is then emitted. If $\mathbf{S}(e + 1) \neq \mathbf{S}(e)$, this event triggers DPSS resharing and epoch activation; otherwise, resharing is skipped and the committee continues unchanged.

The greedy selection procedure is not formally optimal: with multiple interacting per-class constraints, the problem becomes a combinatorial subset-selection task for which greedy strategies cannot guarantee achieving a full size- N_{target} solution even when one exists. We adopt it because it is transparent, auditable, efficient for on-chain execution, and compatible with decentralized trust models. The algorithm naturally generalizes to additional diversity attributes (e.g., physical host, rack, geographic region) by adding corresponding counters and fraction bounds.